DP Optimizations

Convex hull trick, Li Chao tree, divide & conquer DP, Knuth

Learning goals

- Turn a transition into **lines** and query their lower envelope
- Implement the monotone **CHT** and the **Li Chao tree**
- Use the **quadrangle inequality** to justify monotone optimal splits
- Implement **divide & conquer DP** in $O(k\, n \log n)$
- Implement **Knuth's optimization** in $O(n^2)$ and prove the bound

Three slow recurrences

```
dp[i]    = min_{j<i}      dp[j] + cost(j, i)                  O(n²)
dp[c][i] = min_{j<i}      dp[c-1][j] + cost(j, i)             O(k n²)
dp[i][j] = min_{i<=m<j}   dp[i][m] + dp[m+1][j] + C(i, j)     O(n³)
```

The speed-up depends on a **structural property of the cost**.

Property	Tool	Result
cost = product of j-term and i-term	CHT / Li Chao	O(n) / O(n log C)
optimal j monotone in i	D&C DP	O(k n log n)
interval split monotone	Knuth	O(n²)

Motivating example — Monster Game I (CSES 2084)

Kill monster i with skill factor f: time s[i]·f, then factor becomes f[i].

```
dp[i] = min_{j<i}  dp[j] + s[i] · f[j]        (f[0] = x, dp[0] = 0)
```

For fixed j: `f[j] · X + dp[j]` is a **line**.
dp[i] = minimum over all lines so far, at X = s[i].

n ≤ $2 \cdot 10^5$ → need better than $O(n^2)$.

Spotting lines: expand

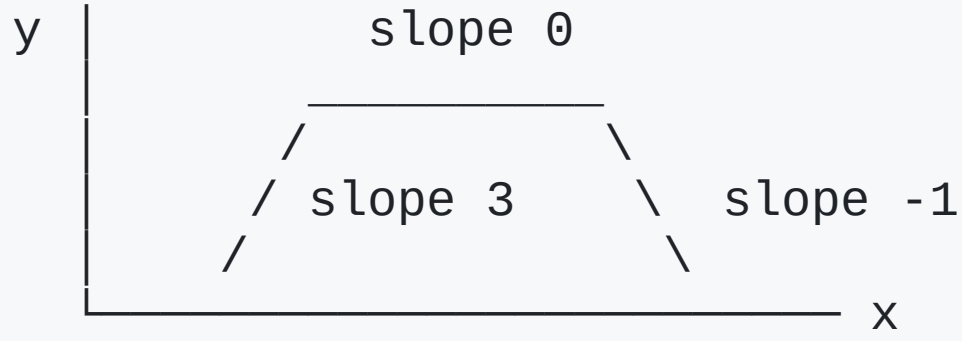
```
dp[i] = min_j dp[j] + (S[i] − S[j])²
      = S[i]²  +  min_j ( −2·S[j] · S[i]  +  dp[j] + S[j]² )
                          ^^^^^^^ slope      ^^^^^^^^^^^^^^ intercept
```

Rule: anything multiplying **both** an i-term and a j-term → slope × x.

Pure i-terms go outside the min; pure j-terms go into the intercept.

The lower envelope

Pointwise min of lines = **concave piecewise-linear** function.



1. Each line is on the envelope on **one interval** (or none)
2. Left → right, slopes **decrease** (for min)

⇒ Add lines in decreasing slope order to a **stack**.

Maintaining the hull

New line c, top b, below it a: b is useless if $x(a \cap c) \le x(a \cap b)$.

```
static bool bad(ll ka, ll ma, ll kb, ll mb, ll kc, ll mc) {
    return (__int128)(mc - ma) * (ka - kb)
        <= (__int128)(mb - ma) * (ka - kc);
}
void add(ll k, ll m) {
    while (K.size() >= 2 && bad(K[K.size()-2], M[M.size()-2],
                                K.back(), M.back(), k, m)) {
        K.pop_back(); M.pop_back();
    }
    K.push_back(k); M.push_back(m);
}
```

Each line pushed/popped once → amortized **O(1)**.

Querying the hull

- **Arbitrary x:** binary search the first i with line i ≤ line i+1 at x — O(log n)
- **Monotone x:** pointer only moves forward — amortized O(1)

```
ll query(ll x) const {
    int lo = 0, hi = (int)K.size() - 1;
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        if (K[mid]*x + M[mid] <= K[mid+1]*x + M[mid+1]) hi = mid;
        else lo = mid + 1;
    }
    return K[lo] * x + M[lo];
}
```

Max instead of min: negate everything.

Li Chao tree — idea

Segment tree over the **x-domain**. Each node keeps **one line**: the best at its midpoint among lines pushed to it.

Insert `nw` at node with line `cur`:

1. If `nw` wins at mid → swap
2. Loser can win on **at most one half** (lines cross once)
3. Compare at `lo`: loser wins at lo → recurse left, else right

Query x: min over lines on the root→leaf path of x.

Li Chao tree — code

```
void add(Line nw, int v, int lo, int hi) {          // range [lo, hi)
    int mid = (lo + hi) / 2;
    bool lef = nw.eval(lo) < t[v].eval(lo);
    bool md  = nw.eval(mid) < t[v].eval(mid);
    if (md) swap(t[v], nw);
    if (hi - lo == 1) return;
    if (lef != md) add(nw, 2 * v, lo, mid);
    else           add(nw, 2 * v + 1, mid, hi);
}
ll query(ll x, int v, int lo, int hi) {
    ll r = t[v].eval(x);
    if (hi - lo == 1) return r;
    int mid = (lo + hi) / 2;
    return min(r, x < mid ? query(x, 2*v, lo, mid)
                          : query(x, 2*v+1, mid, hi));
}
```

Li Chao — why it works

Invariant: the best line at any x lies on x's root→leaf path.

- Kept line wins at mid
- Discarded half: the crossing point is in the *other* half,
  so the kept line wins on the entire discarded half

Cost: O(log C) add & query · segments: O(log² C) · mergeable (CF 932F)

Empty line `{0, INF}`: slope 0 ⇒ evaluating never overflows.

CHT vs Li Chao

	Monotone CHT	Li Chao
Slopes sorted?	required	any order
Queries	O(1) monotone / O(log n)	O(log C)
Segments	no	O(log² C)
Precision	`__int128` in `bad`	only evaluations
Code	~20 lines	~25 lines

Monster Game I → either. Monster Game II (CSES 2085) → Li Chao.

Divide & conquer DP

Layered partition DP: `dp[c][i] = min_{j<i} dp[c-1][j] + cost(j, i)`.

If **opt(i) ≤ opt(i+1)**:

```
solve(lo, hi, optLo, optHi):
    mid = (lo+hi)/2;  scan j ∈ [optLo, min(mid-1, optHi)] → opt(mid)
    solve(lo, mid-1, optLo, opt(mid))
    solve(mid+1, hi, opt(mid), optHi)
```

Depth O(log n); each level scans O(n) total ⇒ **O(n log n) per layer**.

D&C — code (CSES 2086 Subarray Squares)

```
void solve(int lo, int hi, int optLo, int optHi) {
    if (lo > hi) return;
    int mid = (lo + hi) / 2;
    pair<ll, int> best = {LLONG_MAX, -1};
    for (int p = optLo; p <= min(mid - 1, optHi); p++)
        if (prv[p] != LLONG_MAX)
            best = min(best, {prv[p] + cost(p, mid), p});
    cur[mid] = best.first;
    int opt = best.second == -1 ? optLo : best.second;
    solve(lo, mid - 1, optLo, opt);
    solve(mid + 1, hi, opt, optHi);
}
// for each layer: solve(1, n, 0, n - 1); swap(prv, cur);
```

Quadrangle inequality

For all a ≤ b ≤ c ≤ d:

```
cost(a, c) + cost(b, d)  ≤  cost(a, d) + cost(b, c)
     "crossing"          ≤        "nesting"
```

Theorem. QI ⇒ smallest optimal j is non-decreasing in i.

Example: cost = (S[i] − S[j])², S non-decreasing:
LHS − RHS = 2(S_d − S_c)(S_a − S_b) ≤ 0 ✓

Proof sketch (QI ⇒ monotone opt)

$i < i'$, $j = \text{opt}(i)$, $j' = \text{opt}(i')$, suppose **$j' < j$** (smallest minimizers).

QI on (j', j, i, i'), add dp terms:
$f_i(j') + f_{i'}(j) \le f_{i'}(j') + f_i(j)$

j' optimal for $i' \Rightarrow f_{i'}(j') \le f_{i'}(j) \Rightarrow$ **$f_i(j') \le f_i(j)$**

So $j' < j$ is optimal for i — contradicts minimality of j. ∎

Contest shortcut: brute force, **print opt(i)**, check it is sorted.

Knuth's optimization

```
dp[i][j] = min_{i<=m<j} dp[i][m] + dp[m+1][j] + C(i, j)
```

If C satisfies QI **and** C(b,c) ≤ C(a,d) for [b,c] ⊆ [a,d] (Yao 1980):

```
opt[i][j-1]  ≤  opt[i][j]  ≤  opt[i+1][j]
```

C(i, j) = sum(a[i..j]), a ≥ 0 → both hold.

Examples: optimal BST, merging piles, CSES 2088 *Knuth Division*.

Knuth — code

```
for (int i = 1; i <= n; i++) opt[i][i] = i;
for (int len = 2; len <= n; len++)
    for (int i = 1; i + len - 1 <= n; i++) {
        int j = i + len - 1;
        dp[i][j] = LLONG_MAX;
        int from = opt[i][j-1], to = min(opt[i+1][j], j - 1);
        for (int m = from; m <= to; m++) {
            ll v = dp[i][m] + dp[m+1][j] + (S[j] - S[i-1]);
            if (v < dp[i][j]) { dp[i][j] = v; opt[i][j] = m; }
        }
    }
```

Why Knuth is $O(n^2)$

Fixed length L, j = i + L − 1:

```
Σ_i ( opt[i+1][j] − opt[i][j−1] + 1 )
```

Consecutive terms cancel (telescoping) ⇒ O(n) per length.
n lengths ⇒ **$O(n^2)$** total.

Memory: two n×n tables — n = 5000 ≈ 300 MB. Check the limit.

Which optimization?

You see	Try
`f(j) · g(i)` after expanding	CHT / Li Chao
split into k contiguous groups, convex cost	D&C DP
interval DP, cost = interval sum	Knuth
exactly k groups, k large	Aliens trick (WF-09)
window j ≥ i − L	monotone queue (regional-05)

Complexity summary

Technique	Before	After
Monotone CHT	$O(n^2)$	$O(n)$ amortized
CHT + binary search	$O(n^2)$	$O(n \log n)$
Li Chao	$O(n^2)$	$O(n \log C)$
D&C DP	$O(k\, n^2)$	$O(k\, n \log n)$
Knuth	$O(n^3)$	$O(n^2)$

Common mistakes

- Wrong slope order / hull direction (min vs max)
- Equal slopes in monotone CHT — dedupe first
- 64-bit overflow in intersection test → `__int128`
- Li Chao query outside the domain
- Assuming monotone opt without checking QI (or printing opt)
- D&C: allowing $j \geq i$; adding to an INF state
- Knuth: wrong base opt; loop by i instead of by length

Practice problems

- CHT/Li Chao: CSES 2084, 2085, SPOJ ACQUIRE, CF 319C, CF 1083E, CF 631E, CF 932F
- D&C: CSES 2086, 2087, CF 321E, CF 868F, APIO 2014 Sequence
- Knuth: CSES 2088

Summary

- Lines from transitions; lower envelope is concave, slopes sorted
- Monotone CHT: stack, `__int128` test, pointer or binary search
- Li Chao: one line per node, loser goes to one half — always works
- QI ⇒ monotone opt ⇒ D&C DP in $O(k\, n \log n)$
- QI + inclusion-monotone ⇒ Knuth in $O(n^2)$
- When unsure: brute force and **print opt**
- Next: **Lecture 4 — HLD, centroid decomposition, small-to-large**